# Unidad N° 7

Recursos para seguridad y prueba de las aplicaciones web

# Recursos para seguridad

*Introducción*

- En las redes circulan datos sensibles que pueden estar expuestos cuando son enviados entre clientes y servidores.
- Así, dada su importancia, los datos y las aplicaciones web que los procesan pueden sufrir ataques:
  - Ataques pasivos: intercepción de mensajes o de la identidad de usuarios.
  - Ataques activos: denegación de servicio e imposibilidad de acceso a recursos.

# Recursos para seguridad

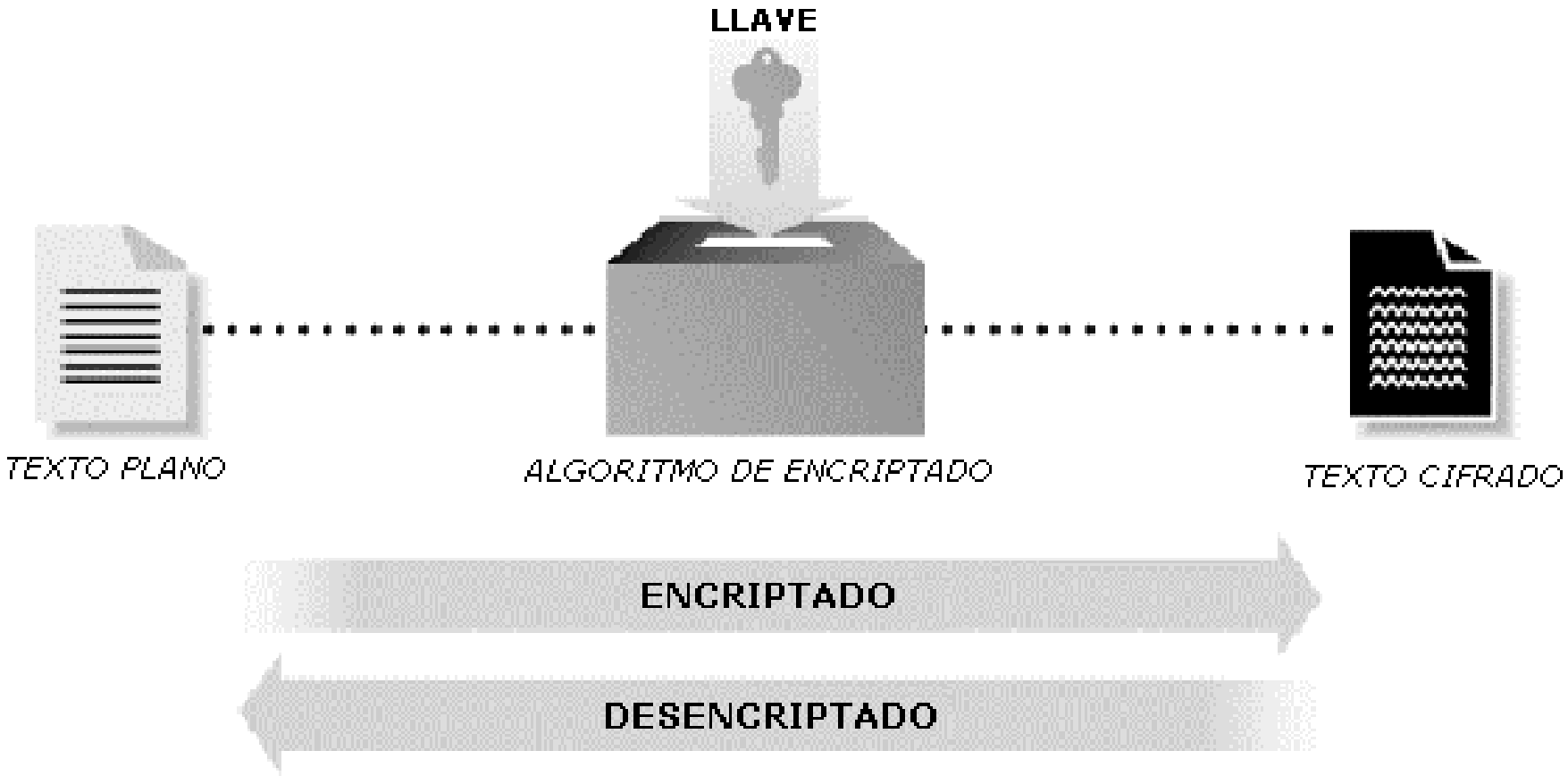
- Por esto, es necesario implementar medidas de seguridad, cuyos objetivos son:
  - Evitar que intrusos lean o manipulen el contenido o la secuencia de mensajes intercambiados en la red sin ser detectados → Algoritmos de encriptación
  - Impedir la falsificación de mensajes por parte de usuarios con intenciones dudosas → Funciones hash
  - Asegurar el acceso al servicio y brindar el mismo nivel a todos los usuarios.
  - Satisfacer los requerimientos legales en resolución de conflictos, protección al consumidor, protección de privacidad y explotación de los datos recolectados de clientes para propósitos comerciales.

# Recursos para seguridad

*Algoritmos de encriptación*

- La criptografía permite transformar un texto en otro, de forma tal que sea incomprensible para receptores no autorizados.
- Los sistemas de encriptación realizan dos procesos:
  - Encriptado: el texto plano se transforma en un texto cifrado mediante un algoritmo de encriptación y una clave de encriptación.
  - Desencriptado: el texto cifrado se convierte en el texto plano original mediante un algoritmo de encriptación y una clave de desencriptación.

# Recursos para seguridad



# Recursos para seguridad

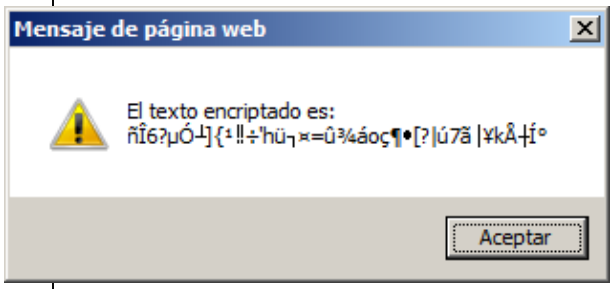
- Los tipos de encriptación pueden clasificarse en:
  - Encriptación simétrica: los algoritmos de claves simétricas utilizan la misma clave secreta para encriptar y desencriptar el mensaje.
    - Ejemplos: AES (Advanced Encryption Standard), DES (Data Encryption Standard), IDEA (International Data Encryption Algorithm), RC2, RC4, RC5, 3DES.
  - Encriptación con clave pública: los algoritmos de clave pública utilizan una para encriptar el mensaje (clave pública) y otra para desencriptarlo (clave privada).
    - Ejemplo: RSA.
- Tanto PHP como JavaScript soportan el uso de los algoritmos de encriptación. PHP lo hace a través de la biblioteca Mcrypt.

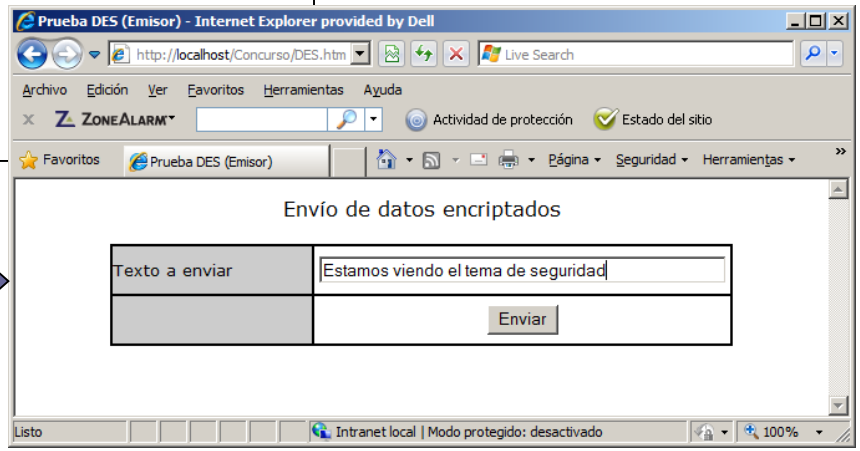
# Recursos para seguridad

## *Ejemplo: Encriptación del texto con JavaScript*

```html
<html>
<head>
<title>Prueba DES (Emisor)</title>
<script language="javascript" src="DES.js">
</script>
</head>

<body>
<section>
<article>
<p>Env&iacute;o de datos encriptados</p>
<table width="500" border="1" cellpadding="0" cellspacing="0" bordercolor="#000000">
  <tr>
    <td width="160" height="40" bgcolor="#CCCCCC">Texto a enviar</td>
    <td height="40" align="center"><input name="texto" type="text" id="texto" size="50" maxlength="100"></td>
  </tr>
  <tr>
    <form action="DES.php" method="post" name="formulario" id="formulario">
    <td width="160" height="40" bgcolor="#CCCCCC">
    <input name="textoEncriptado" type="hidden" id="textoEncriptado">
    </td>
    <td height="40" align="center">
    <input name="Enviar" type="button" id="Enviar" onClick="EnvioTextoEncriptado(texto, formulario);" value="Enviar">
    </td>
    </form>
  </tr>
</table>
</article>
</section>
</body>
</html>
```

```javascript
function EnvioTextoEncriptado(ptext, pform)
{
    var vpassword = "claveEncriptadoTripleDES";
    var encriptado;
    if(ptext.value == "")
    {   alert('Error. No se ingresó el texto');
        ptext.focus();
    }
    else
    {   encriptado = des(vpassword, ptext.value, 1);
        alert("El texto encriptado es: " + encriptado);
        pform.textoEncriptado.value = stringToHex(encriptado);
        pform.submit();
    }
}
```

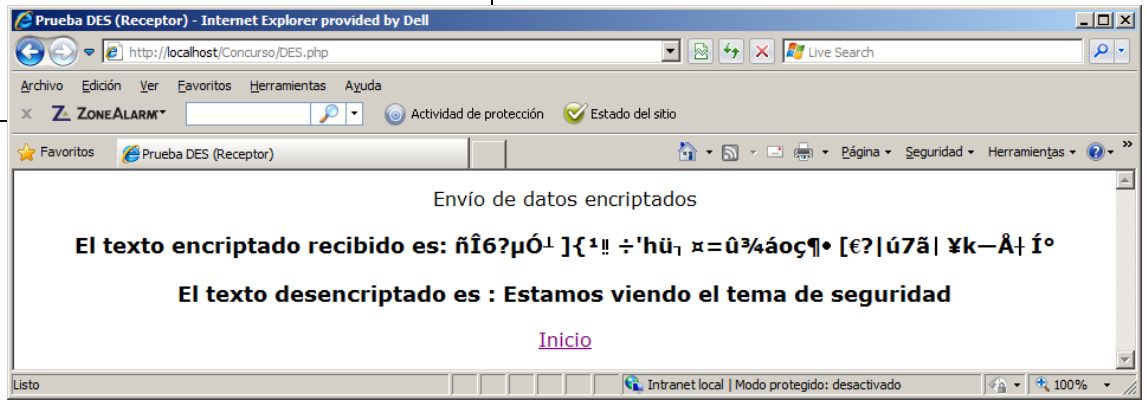


# Recursos para seguridad

## *Ejemplo: Desencriptación del texto con PHP*

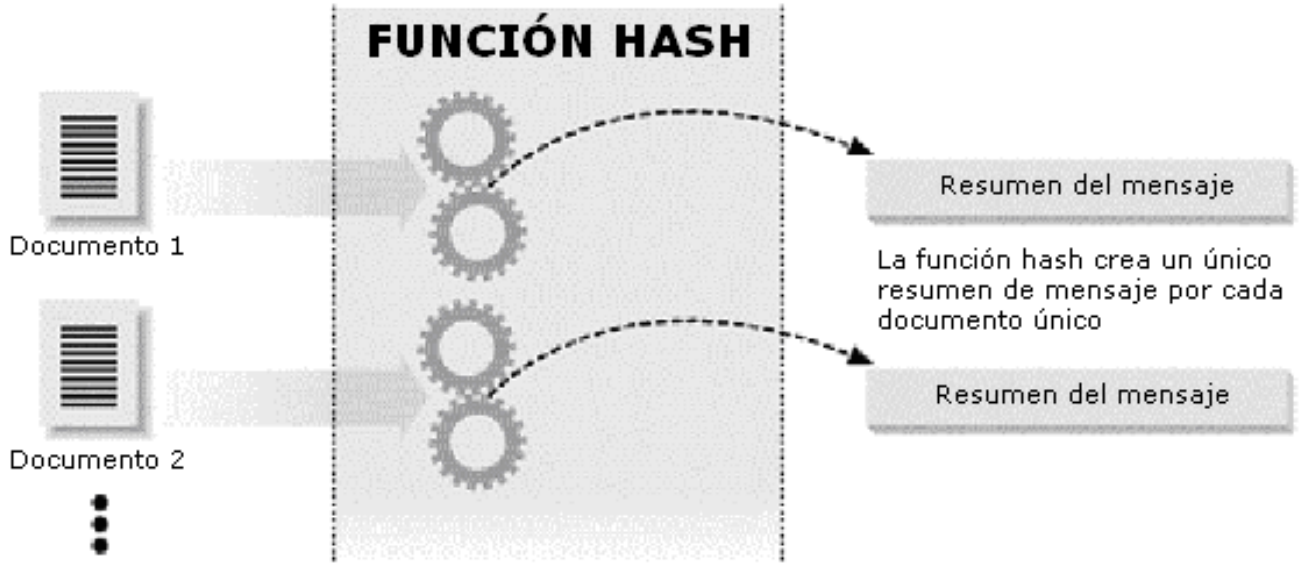
```php
<html>
<head>
<title>Prueba DES (Receptor)</title>
</head>

<body>
<section>
<article>
<?php
include_once("DES.inc.php");
echo '<p>Env&iacute;o de datos encriptados</p>';
if (is_null($_POST['textoEncriptado']))
{   //No se recibió el texto encriptado. muestro mensaje de error
    echo '<h2>Error. No se recibió el texto encriptado</h2>';
}
else
{   //Se recibió el texto encriptado. Lo desencripto usando la clave
    $textoRecibido = hexToString($_POST['textoEncriptado']);
    echo '<h3>El texto encriptado recibido es: '.$textoRecibido.'</h3>';
    echo '<h3>El texto desencriptado es : '.des(clave,$textoRecibido,0).'</h3>';
}
echo '<a href="DES.htm" target="_self">Inicio</a>';
?>
</article>
</section>
</body>
</html>
```



# Recursos para seguridad

*Funciones hash*

- Una función hash es un recurso que permite asegurar la integridad de los datos:
  - Se aplica sobre un mensaje y devuelve un resumen (código alfanumérico) único del mensaje.



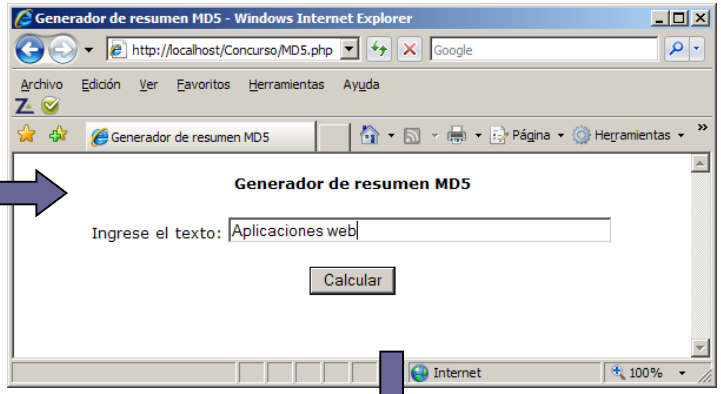
# Recursos para seguridad

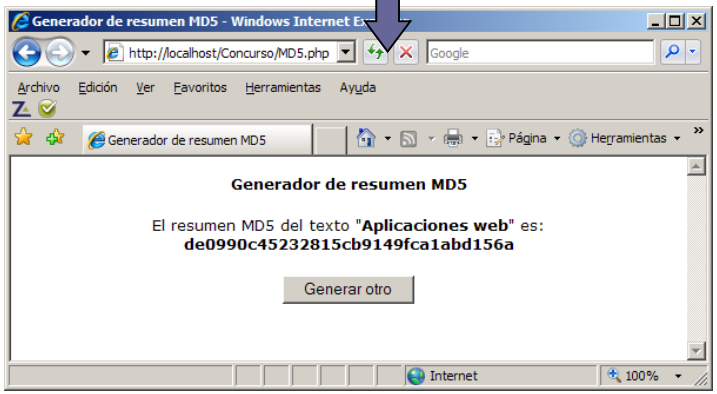
- Algunas características de las funciones hash son:
  - No produce colisiones: no se obtiene el mismo resumen para dos documentos diferentes.
  - Es imposible de invertir: no se puede determinar el mensaje a partir de su resumen.
  - Produce muchos cambios en el resumen: si el documento cambia, el resumen también.
- Ejemplos de funciones hash: MD5 (Message Digest Algorithm), RIPEMD, SHA y SHA1.
- Tanto PHP como JavaScript soportan el uso de funciones hash. PHP incorpora las funciones MD5 y MD5_file, que permiten obtener el resumen de una cadena o un archivo respectivamente.

# Recursos para seguridad

## *Ejemplo: Resumen MD5 en PHP*

```
1  <html>
2  <head>
3  <title>Generador de resumen MD5</title>
4  <script language="JavaScript" type="text/JavaScript">
31 </head>
32 <body>
33 <section>
34 <article>
35 <?php
36 if (isset($_POST['boton']) && ($_POST['boton']=="Calcular"))
37 {
38 ?>
39 <form name="form1" method="post" action="MD5.php">
40   <p><strong>Generador de resumen MD5</strong></p>
41   <p>El resumen MD5 del texto "<b><?php echo $_POST['texto']; ?></b>" es:
42   <b><?php echo md5($_POST['texto']);?></b></p>
43   <p><input name="boton" type="submit" id="boton" value="Generar otro"></p>
44 </form>
45 <?php
46 }
47 else
48 {
49 ?>
50 <form action="MD5.php" method="post" name="formulario" id="formulario"
51     onSubmit="MM_validateForm('texto','','R');return document.MM_returnValue">
52     <p><strong>Generador de resumen MD5</strong></p>
53     <p>Ingrese el texto:
54     <input name="texto" type="text" id="texto" size="50" maxlength="100">
55     </p>
56     <p><input name="boton" type="submit" id="boton" value="Calcular"></p>
57 </form>
58 <?php
59 }
60 ?>
61 </article>
62 </section>
63 </body>
64 </html>
```



# Recursos para seguridad

*Algunas políticas de seguridad para el envío de passwords*

| Datos enviados | | En la BD | En el servidor |
|---|---|---|---|
| 1 | usuario y password limpio | password limpio | Comparar los password |
| 2 | usuario y password encriptado | password limpio | Desencriptar el password recibido y comparar con el almacenado en la BD |
| 3 | usuario y password limpio | resumen del password | Calcular el resumen del password recibido y comparar con el almacenado en la BD |
| 4 | usuario y resumen del password | resumen del password | Comparar los resúmenes |
| 5 | usuario y resumen con sal del password | resumen del password | Calcular los resúmenes con sal y comparalos |

Seguridad

¿Qué opciones existen para recuperar un password?

# Ataques a aplicaciones Web

*Ataques a las aplicaciones web*

- Además de implementar los recursos para seguridad, una aplicación web debe estar preparada para otros ataques, como ser:
  - SQL Injection
  - Cross-Site Scripting
  - Phishing
  - Session Hijacking

# Ataques a aplicaciones Web

*SQL Injection*

- Los datos ingresados en los formularios a veces son utilizados para realizar consultas a la base de datos.
- Esto puede ser peligroso si no se validan las entradas de datos sensibles.
- Así, se puede producir un error en ejecución que se mostrará en el navegador, y puede divulgar la estructura de la base de datos al atacante.
- O también, un usuario podría pasar el control de identificación, o ingresar una sentencia SQL que permitiría borrar tablas o la base de datos.

# Ataques a aplicaciones Web

*Ejemplos*

| | |
|---|---|
| Consulta en el script | $consulta = “SELECT titulo FROM libros WHERE codigo = ”.$_POST[‘codigo’]; |
| Valor ingresado | codigo = 23; DROP TABLE libros |
| Consulta a ejecutar | SELECT titulo FROM libros WHERE codigo = 23; DROP TABLE libros; |

| | |
|---|---|
| Consulta en el script | $consulta = “SELECT id FROM usuarios WHERE usuario = ‘”.$_POST[‘usuario’].“’AND pass = ‘ ”.$_POST[‘clave’].“ ’ ”; |
| Valores ingresados | usuario = juan – pass = ‘OR ‘1 |
| Consulta a ejecutar | SELECT id FROM usuarios WHERE usuario = ‘juan’ AND pass = ‘’ OR ‘1’; |

# Ataques a aplicaciones Web

## *Ejemplo de vulnerabilidad*

http://www.aeropuertosarg.com.ar/aeropuertos/ - Microsoft Internet Explorer

Archivo Edición Ver Favoritos Herramientas Ayuda

Atrás

Dirección http://www.aeropuertosarg.com.ar/aeropuertos/ Ir

**Warning**: mysql_connect() [function.mysql-connect]: Can't connect to local MySQL server through socket '/var/lib/mysql/mysql.sock' (2) in **/home/aeropuer/public_html/aeropuertos/base/conectar.inc** on line **4**

Listo Internet

http://www.aeropuertosarg.com.ar/aeropuertos/base/conectar.inc - Internet Explorer provided by Dell

http://www.aeropuertosarg.com.ar/aeropuertos/base/conectar.inc Live Search

Archivo Edición Ver Favoritos Herramientas Ayuda

Favoritos http://www.aeropuertosarg.com.ar/aeropuertos/bas... Página Seguridad Herramientas

```
<?php
function conectar()
{
    if(!($link=mysql_connect("localhost","aeropuer","H4dFwO8uB")))
    {
        exit();
    }

    mysql_query("SET character_set_results='latin1'");
    if(!(mysql_select_db("aeropuer_aeropuertos",$link)))
    {
        exit();
    }

    return $link;
}

?>
```

Listo Internet | Modo protegido: activado 100%

# Ataques a aplicaciones Web

- Algunas formas de evitar este ataque son:
  - Validar todos los datos que se ingresan a la base de datos, incluyendo los tipos de datos esperados.
  - Verificar como trabajan las consultas SQL que se incluyen en el script PHP.
  - De ser posible, utilizar procedimientos almacenados.
  - Otorgar a los usuarios sólo los permisos de acceso que necesita.
  - Si se trabaja con librerías que incluyen los datos de la conexión (usuario, password, servidor, base de datos), guardarla en un directorio inaccesible por los clientes y tener precaución con la extensión del archivo, ya que podría ser visualizado.

# Ataques a aplicaciones Web

*Phishing*

- El phishing es el arte de suplantar sitios web, utilizando la replicación del mismo, incluyendo hasta los formularios de entrada de datos.
- Luego, al usuario se lo puede engañar para que facilite sus datos, sin que él sepa que no esta trabajando con el sitio web auténtico.
- Otra forma es enviando un correo electrónico con un enlace hacia el sitio replicado, donde se le solicitarán nuevamente los datos.
- Una forma de evitar este ataque es usando certificados válidos otorgados por autoridades certificantes.

# Ataques a aplicaciones Web

*Ejemplo*

SquirrelMail 1.4.22

www.ing.unp.edu.ar/webmail/src/webmail.php

**Carpetas**
Última actualización:
Vie, 10:49 pm
(Comprobar correo)

- ENTRADA (4)
  - Drafts
  - Sent
  - Trash (Purgar)

Carpeta actual: **ENTRADA** — Desconectarse

Componer | Direcciones | Carpetas | Opciones | Buscar | Ayuda — Facultad de Ingenieria

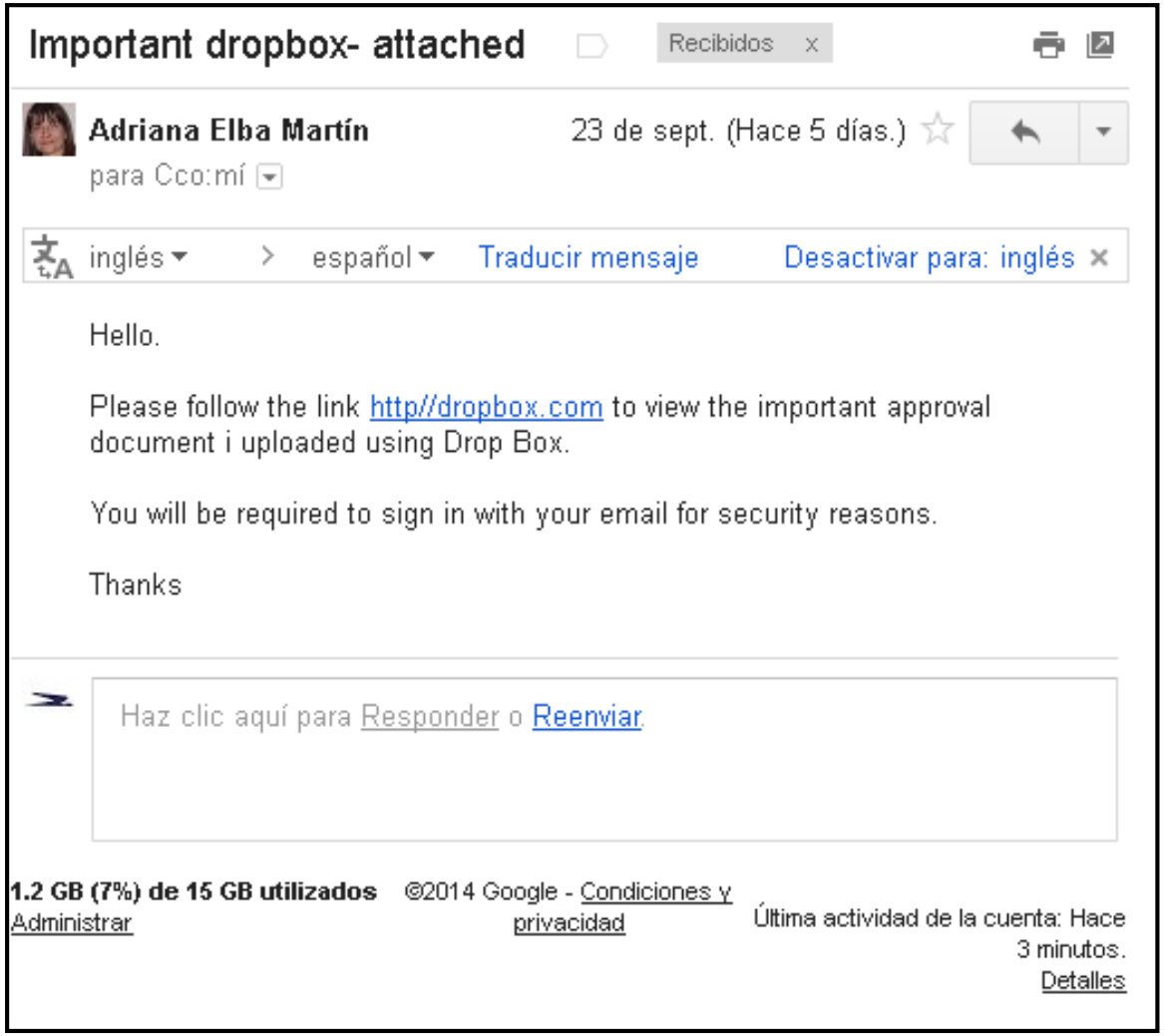
Lista de mensajes | No leído | Borrar — Anterior | Siguiente — Reenviar | Reenviar como adjunto | Responder | Responder a todos

**Asunto:** advertencia
**De:** "Webmaster" <webmaster@ondomain.com>
**Fecha:** Mie, 10 de Septiembre de 2014, 10:57 am
**Para:** "Recipient" <webmaster@ondomain.com>
**Prioridad:** Normal
**Opciones:** Ver encabezado completo | Ver versión imprimible | Bajar este mensaje como un archivo

```
Su buzón ha superado el límite de almacenamiento de 2.GB
Establecido por el administrador es actualmente 2,30 GB, no puede
enviar o recibir nuevos mensajes hasta que vuelva a validar su e-mail

Haga clic en el siguiente enlace para validar tu e-mail

http://serviciodecorreo563.tripod.com/servicio/

¡gracias
administrador del sistema
```

# Ataques a aplicaciones Web

*Ejemplo*



Important dropbox- attached

Recibidos x

**Adriana Elba Martín** 23 de sept. (Hace 5 días.)

para Cco:mí

inglés > español Traducir mensaje Desactivar para: inglés ×

Hello.

Please follow the link http//dropbox.com to view the important approval document i uploaded using Drop Box.

You will be required to sign in with your email for security reasons.

Thanks

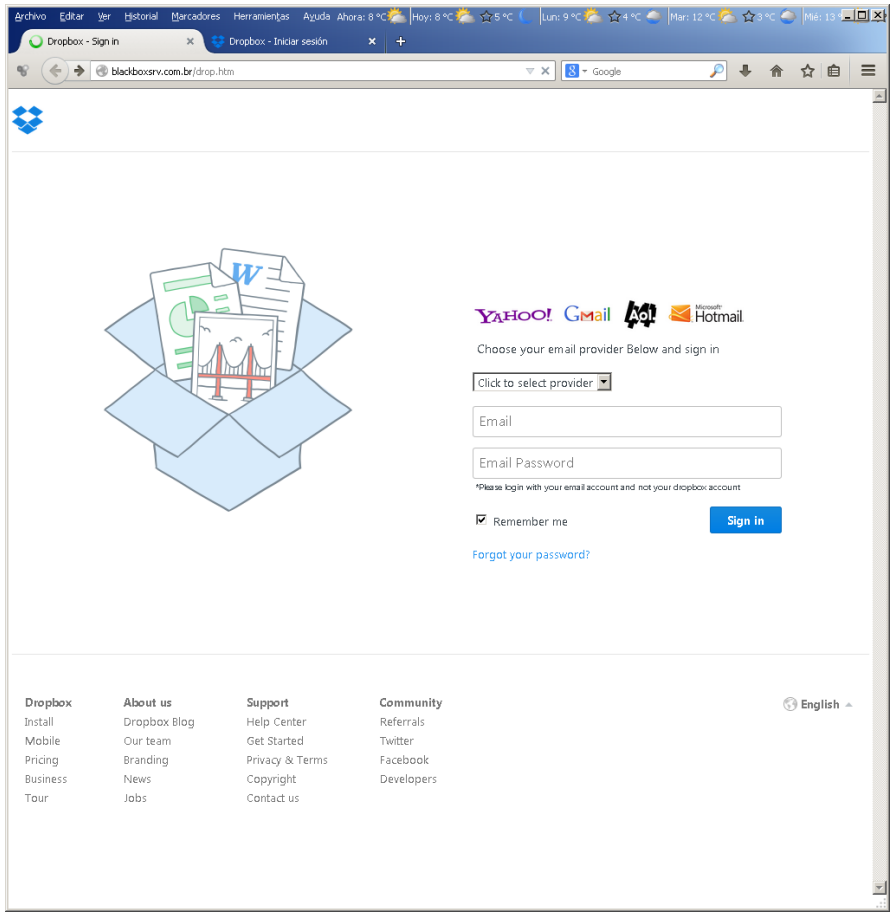
Haz clic aquí para Responder o Reenviar.

**1.2 GB (7%) de 15 GB utilizados** Administrar

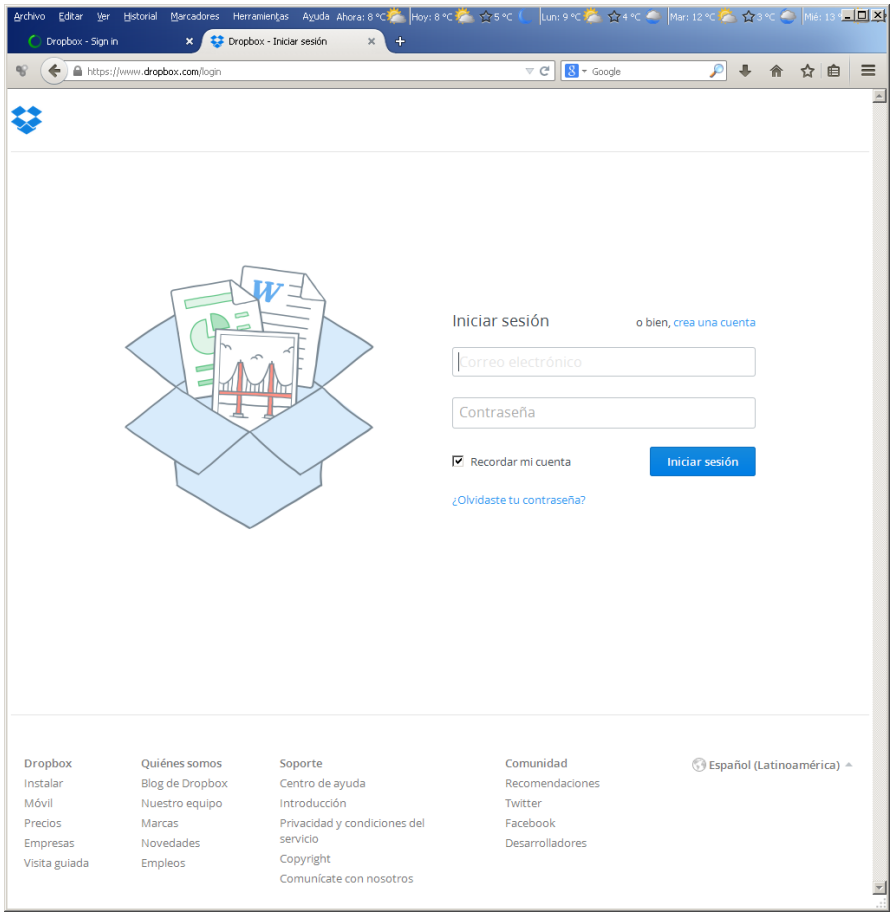
©2014 Google - Condiciones y privacidad

Última actividad de la cuenta: Hace 3 minutos. Detalles

# Ataques a aplicaciones Web




http://blackboxsrv.com.br/drop.htm




https://www.dropbox.com/login

# Ataques a aplicaciones Web

*Session Hijacking*

- Cuando un usuario autenticado está conectado a un sitio, es posible que otro tome el control de su sesión.
- Esto se logra obteniendo el ID de la sesión o bien recuperando la cookie de la misma.
- El ID de la sesión puede obtenerse utilizando técnicas de fuerza bruta o intercepción de mensajes.
- Una forma de evitar este ataque es definiendo tiempos de expiración por inactividad y absoluto.
- Otra forma es usando SSL, para el caso de la intercepción.

# Proceso de prueba

- El proceso de prueba comienza con pruebas que analizan la funcionalidad del contenido y la interfaz (lo primero que ve el usuario).
- Luego se analizan aspectos de la arquitectura del diseño y de la navegación.
- Finalmente, se realizan pruebas que examinan las capacidades tecnológicas, que no siempre son visibles a los usuario (cuestiones de infraestructura e instalación/implementación de la aplicación web).

# Proceso de prueba

- Las pruebas que se pueden realizar sobre una aplicación web están centradas en los siguientes aspectos:
  - Contenido
  - Interfaz
  - Navegación
  - Componentes
  - Rendimiento
  - Configuración
  - Seguridad

# Prueba de Contenido

- La prueba de contenido intenta descubrir errores como:
  - Errores tipográficos
  - Información incorrecta
  - Organización inadecuada
  - Violación de leyes de propiedad intelectual
- Para ello, esta prueba tiene 3 objetivos principales:
  1. Descubrir errores sintácticos (de redacción o gramaticales).
  2. Descubrir errores semánticos (de precisión o completitud de la información).
  3. Encontrar errores de organización o estructura del contenido que se presenta al usuario final.

# Prueba de Contenido

*Prueba de base de datos*

- Una aplicación web puede tener una interfaz con una base de datos y construir objetos de contenido dinámico con datos de la base de datos.
- Para crear éstos objetos de contenido dinámico, se sigue una secuencia como la siguiente:
  1. Se crea una conexión a la base de datos.
  2. Se consulta sobre los datos de interés.
  3. Se organizan los datos obtenidos en la consulta dentro de un objeto de contenido.
  4. Se transmite ese objeto de contenido desde el servidor hacia el cliente que lo solicitó.
- En cualquiera de los pasos pueden ocurrir errores

# Prueba de Contenido

- Así, la prueba de base de datos intenta encontrar esos errores, pero esto es complicado debido a que:
  - La información solicitada por el cliente rara vez se presenta en la forma en la que se ingresa al DBMS → diseñar pruebas para descubrir errores en las consultas a realizar al DBMS.
  - La aplicación web y el DBMS no necesariamente se encuentran en el mismo servidor → desarrollar pruebas de conexión.

# Prueba de Contenido

- Los datos de las consultas deben enviarse al servidor web de la aplicación y formatearse para luego enviarse al cliente → diseñar pruebas que validen los datos recibidos desde el DBMS y pruebas que verifiquen las transformaciones antes de crear los objetos de contenido.
- El objeto de contenido dinámico debe transmitirse al cliente → diseñar pruebas para descubrir errores de formato del objeto y para probar compatibilidad con la configuración del entorno del cliente.

# Prueba de Interfaz

*Estrategia de prueba de interfaz*

- La prueba de interfaz ejercita los mecanismos de interacción y valida los aspectos estéticos de la interfaz de usuario.
- Para esto, sigue una estrategia global:
  - Descubrir errores relacionados con mecanismos de interfaz específicos → ejecución correcta de vinculos o forma de ingreso de datos en formularios.
  - Descubrir errores en la implementación de la semántica de navegación, la funcionalidad de la aplicación web o el despliegue del contenido.

# Prueba de Interfaz

- Para implementar la estrategia, se inician algunos pasos tácticos:
  - Las características de la interfaz se prueban para garantizar que las reglas del diseño, estética y contenido visual relacionado estén disponibles sin error para el usuario → fuentes, uso de colores, imágenes, tablas, etc.
  - Los mecanismos de interfaz individuales se prueban de forma análoga a la prueba de unidad → scripts del cliente, contenido de streaming, carrito de compras, etc.
  - Cada mecanismo de interfaz se prueba dentro del contexto de un caso de uso para un usuario específico → prueba de integración

# Prueba de Interfaz

- La interfaz completa se prueba contra los casos de uso seleccionados a fin de descubrir errores de semántica de la interfaz → prueba de validación
- La interfaz se prueba dentro de varios entornos para garantizar su compatibilidad → prueba de configuración.

# Prueba de Interfaz

*Prueba de mecanismos de interfaz*

- La interacción entre un usuario y la aplicación se logra a través de uno o más mecanismos de interfaz.
- Así, los mecanismos de interfaz incluyen a los links, formularios, scripts del lado del cliente, ventanas emergentes, contenido de streaming, cookies y los específicos de la aplicación.

# Prueba de Interfaz

| Mecanismo | Pruebas |
|---|---|
| Links | ◻ Listar todos los links, internos y externos, dentro de la interfaz y de cada objeto de contenido y se los ejecutan a cada uno individualmente.<br>■ Los externos deben evaluarse para determinar el riesgo de que dejen de existir. |
| Formularios | ◻ Verificar que las etiquetas identifican correctamente los campos y que los campos obligatorios son visibles para el usuario.<br>◻ Verificar que el servidor recibe todos los datos enviados desde el cliente.<br>◻ Chequear si se utilizan correctamente los valores por defecto cuando el usuario no ingresa datos en combos o listas.<br>◻ Verificar que las funciones del browser no corrompen los datos ingresados en el formulario. |

# Prueba de Interfaz

| Mecanismo | Pruebas |
|---|---|
| Formularios (cont.) | □ Chequear que los scripts de validación de datos funcionan correctamente.<br>□ Verificar que los campos del formulario tienen el ancho y tipo adecuado.<br>□ Verificar que los campos de texto no acepten más caracteres de los esperados.<br>□ Los ítems de los combos y listas deben ordenarse de manera significativa para el usuario.<br>□ Verificar que el “autollenado” del browser no produce errores en la entrada de datos del usuario.<br>□ Verificar el orden del foco de los controles del formulario (tecla “Tab”). |

# Prueba de Interfaz

| Mecanismo | Pruebas |
|---|---|
| Scripts del lado del cliente | □ Realizar pruebas de caja negra para encontrar errores en el procesamiento conforme se ejecuta el script.<br>□ Realizar una prueba de compatibilidad del lenguaje de scripts elegido con las configuraciones del entorno de los usuarios. |
| Ventanas emergentes | □ Probar que la aparición instantánea tiene el tamaño y posición adecuada.<br>□ Probar que la aparición no cubre la ventana de la app web.<br>□ La ventana es consistente con el diseño estético de la app web.<br>□ Las barras de desplazamiento se ubican y funcionan bien. |
| Contenido de streaming | □ Verificar que los datos de streaming están actualizados, que se despliegan correctamente y que pueden pausarse y continuar la ejecución sin dificultad |

# Prueba de Interfaz

| Mecanismo | Pruebas |
|---|---|
| Cookies | ◻ Se deben probar tanto del lado del cliente como del lado del servidor.<br>■ Del lado del servidor se debe verificar que la cookie se creo correctamente, con los datos correctos y que se transmitió de manera adecuada cuando se la solicitó. Además se debe verificar su persistencia.<br>■ Del lado del cliente, las pruebas se centran en chequear si la cookie se ligó correctamente con el script que la requiere. |

# Prueba de Interfaz

*Prueba de usabilidad*

- Esta prueba evalúa:
  - el grado con el que los usuarios pueden interactuar efectivamente con la aplicación y
  - el grado con el que la aplicación guía las acciones del usuario, proporciona retroalimentación significativa y refuerza un enfoque de interacción consistente.
- Las pruebas de usabilidad se diseñan para determinar el grado en el que la interfaz facilita la “vida” del usuario.

# Prueba de Interfaz

- La estrategia del diseño de las pruebas de usabilidad se puede plantear de la siguiente manera:
  1. Definir un conjunto de categorías de pruebas de usabilidad e identificar las metas de cada una.
  2. Diseñar pruebas que permitirán la evaluación de cada meta.
  3. Seleccionar a los participantes que realicen las pruebas.
  4. Instrumentar la interacción de los participantes con la aplicación web mientras se lleva a cabo la prueba.
  5. Desarrollar un mecanismo para valorar la usabilidad de la aplicación.

# Prueba de Interfaz

- Las categorías del paso 1 de la estrategia pueden ser:
  - Interactividad → ¿los mecanismos de interacción son fáciles de entender y usar?
  - Plantilla → ¿los mecanismos de navegación, contenido y funciones se ubican de manera tal que pueden encontrarse fácilmente?
  - Legibilidad → ¿el texto está bien escrito y es comprensible? ¿las imágenes son representativas?
  - Estética → ¿la plantilla, color, fuente y características relacionadas facilitan el uso? ¿los usuarios se sienten cómodos con la aplicación?

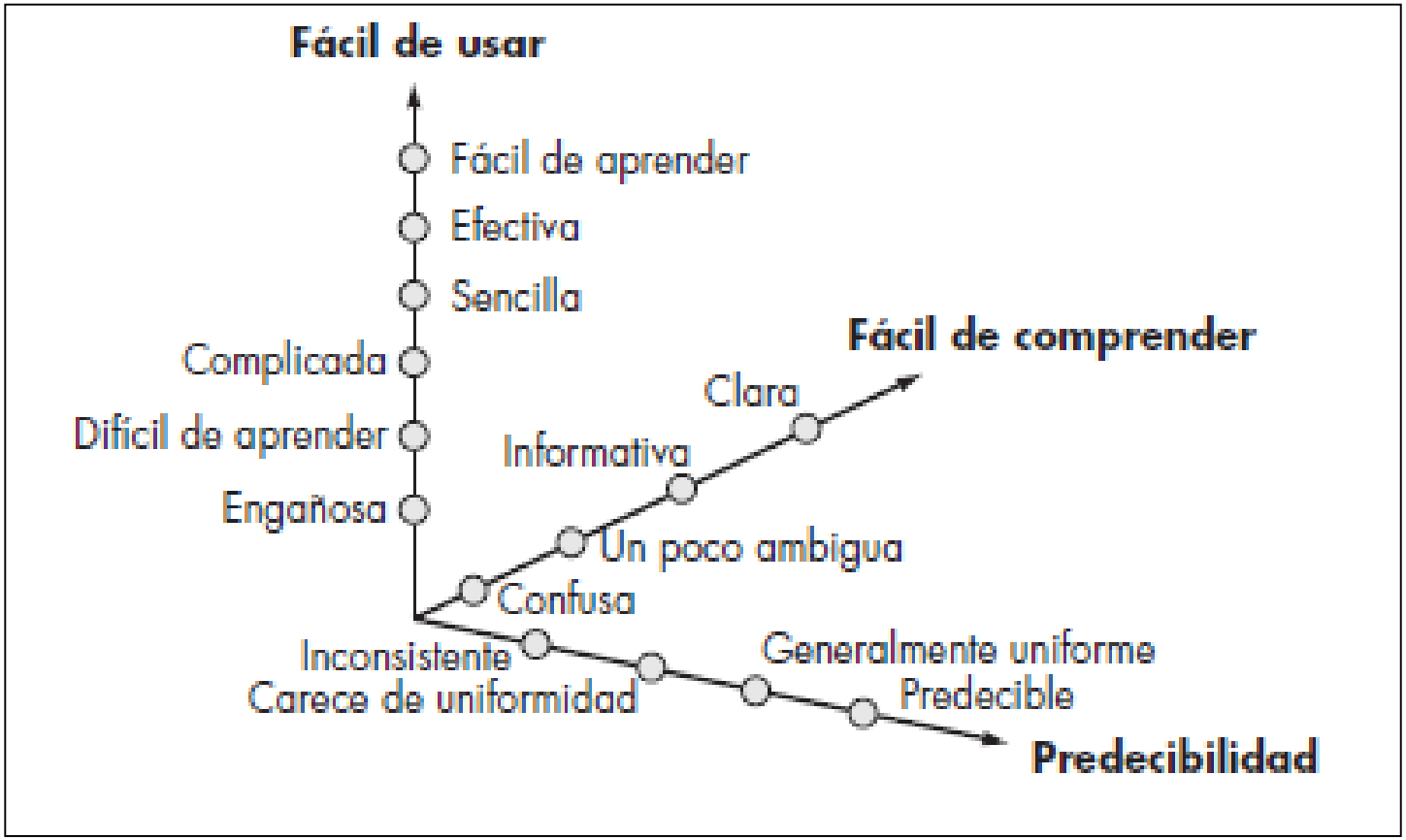
# Prueba de Interfaz

- Características de despliegue → ¿la aplicación usa de manera óptima el tamaño y resolución de la pantalla?
- Sensibilidad temporal → ¿las características, funciones y contenido importantes pueden usarse o adquirir en forma oportuna?
- Personalización → ¿la aplicación se adapta a las necesidades específicas de diferentes categorías de usuario o usuarios individuales?
- Accesibilidad → ¿la aplicación web es accesible a personas que tienen discapacidades?

- Así, dentro de cada una de éstas categorías se diseñan una serie de pruebas relacionadas con la usabilidad.

# Prueba de Interfaz

*Valoración de usabilidad para los mecanismos de interacción e interfaz*



# Prueba de Interfaz

*Prueba de compatibilidad*

- Esta prueba busca descubrir problemas en la ejecución de la aplicación debido a diferentes configuraciones del entorno, producidas por:
  - Arquitecturas de computadoras
  - Sistemas operativos
  - Navegadores
  - Velocidades de conexión
- Así, la configuración del entorno del cliente puede producir diferencias en: las velocidades de procesamiento en el lado del cliente, la resolución de despliegue, las velocidades de conexión, etc.

# Prueba de Interfaz

- Los navegadores también pueden producir que la aplicación se muestre de manera diferente en cada uno de ellos.
- Para evitar éstos problemas, deben realizarse pruebas de compatibilidad.
- El primer paso es definir un conjunto de configuraciones de cómputo, y sus variantes, de acuerdo a los usuarios de la aplicación.
- Luego, se puede crear una estructura de árbol y se identifican las plataformas de cómputo, sistemas operativos, navegadores, velocidades de conexión, etc.

# Prueba de Interfaz

- Luego se realizan las pruebas de validación de compatibilidad (que pueden ser derivadas de pruebas de interfaz, de navegación, de rendimiento y de seguridad).
- La intención es descubrir errores que puedan rastrearse para identificar diferencias de configuración.

# Prueba de Navegación

- Los usuarios navegan siempre dentro de la aplicación web.
- Tienen muchas alternativas para recorrerla y la ruta es predecible cuando tienen un conjunto de objetivos definidos, o puede ser impredecible cuando el usuario encuentra algo inusual dentro de su objetivo.
- Por esto, es importante una prueba de navegación.
- Básicamente hay 2 pruebas de navegación, que evalúan la sintaxis y la semántica.

# Prueba de Navegación

*Prueba de sintaxis de navegación*

- Los mecanismos de navegación se prueban para verificar que cada interfaz realiza la función asignada.

| Mecanismo | Pruebas |
|---|---|
| Links de navegación | ◻ Se prueban los links internos de la aplicación, externos hacia otras aplicaciones y anclas dentro de una página web específica. |
| Redirecciones | ◻ Se utilizan cuando un usuario solicita una URL inexistente, o cuando selecciona un link hacia un contenido removido o cuyo nombre cambió. Se prueba entonces que se despliegue un mensaje aclaratorio y se redireccione hacia otro lado correcto. |

# Prueba de Navegación

| Mecanismo | Pruebas |
|---|---|
| Marcadores de página | ◻ Si bien son funciones del browser, la aplicación web debe probarse para garantizar la extracción correcta de un título de página significativo de acuerdo a la marca |
| Mapas del sitio | ◻ Si la aplicación proporciona un mapa con acceso a todos los contenidos, entonces debe probarse cada entrada del mapa para validar que llevan al usuario al contenido o funcionalidad adecuados. |
| Motores de búsqueda internos | ◻ Un motor de búsqueda interno permite al usuario realizar una búsqueda de palabra clave dentro de la aplicación web para encontrar el contenido necesario. La prueba del motor de búsqueda valida la precisión y completitud de la búsqueda, las propiedades de manejo de error del motor y las características de búsqueda avanzada. |

# Prueba de Componente

- Esta prueba realiza evaluaciones para descubrir errores en funciones o en los scripts de las aplicaciones web.
- Cada función de una aplicación web es un componente de software y puede probarse usando técnicas de caja negra o de caja blanca.
- Estas pruebas normalmente se inician a partir de las entradas de los usuarios mediante formularios.
- A partir del ingreso de los datos, se inicia la ejecución de la función.
- Para probar las funciones, se pueden utilizar los siguientes métodos de diseño de casos de prueba:

# Prueba de Componente

*Partición de equivalencia*

- El dominio de entrada de la función se divide en categorías y se plantean casos de prueba para ellas.
- Así, se crean categorías para diferentes situaciones de un valor esperado (como ser: incompleto, incorrecto, inexistente, formato erróneo, etc) y se arman pruebas para cada caso.

*Análisis de valor de frontera*

- Los datos ingresados en los formularios se prueban en sus fronteras: se ingresan los valores límite y los cercanos a ellos y se verifica como reacciona la aplicación para cada valor.

# Prueba de Componente

*Prueba de rutas*

- Esta prueba se puede utilizarse cuando la complejidad lógica de la función es alta.
- La prueba de rutas es un método de diseño de casos de prueba de caja blanca que se utiliza para garantizar que se ejercitó cada ruta independiente en el programa.

*Prueba de error forzado*

- Ésta prueba plantea casos de prueba que a propósito conducen al componente web a una condición de error.

# Prueba de Componente

- El objetivo es descubrir los errores que ocurren durante la manipulación del error (ej: mensajes de error incorrectos, falla de la aplicación como consecuencia del error, etc).

# Prueba de Configuración

- Esta prueba plantea un desafío debido a la variabilidad e inestabilidad de la configuración de los equipos que acceden a la aplicación.
- La tarea de esta prueba no es ejercitar toda configuración posible del lado del cliente, sino un conjunto de probables configuraciones, tanto del cliente como del servidor.

*Conflictos en el lado del servidor*

- Los casos de prueba se diseñan para verificar que la configuración del servidor proyectada puede ejecutar la aplicación web sin problemas.
- Así, cada prueba de configuración debe considerar cada componente de la configuración del servidor.

# Prueba de Configuración

- Algunas preguntas que deben plantearse son:
  - ¿La aplicación web es completamente compatible con el sistema operativo del servidor web?
  - ¿Los archivos de sistema, directorios y datos del sistema relacionados se crean correctamente cuando la aplicación web está operativa?
  - ¿Las medidas de seguridad del sistema permiten la correcta ejecución de la aplicación y atender a los usuarios sin influir en el rendimiento?
  - ¿La aplicación se probó con alguna configuración de servidor distribuido?
  - ¿La aplicación web se integró correctamente con el DBMS? ¿Los cambios de versiones del DBMS afectan a la aplicación?

# Prueba de Configuración

- ¿Los scripts del lado del servidor se ejecutan adecuadamente?
- ¿Los errores del administrador del sistema se examinaron en sus efectos sobre las operaciones de la app web?

*Conflictos en el lado del cliente*

- Las pruebas se centran en la compatibilidad de la aplicación web con las configuraciones que tienen en su conjunto los siguientes componentes:
  - Hardware
  - Sistemas operativos
  - Software navegador (incluyendo plug-ins)
  - Conectividad

# Prueba de Configuración

- Además de éstos componentes, influyen otras variables como ser el software de redes, cuestiones del ISP y aplicaciones concurrentes.
- Luego, las pruebas se diseñan considerando una muestra representativa de las posibles configuraciones de los clientes.
  - Para esto, pueden utilizarse datos del mercado para obtenerse los componentes mas probables.

# Prueba de Seguridad

- Las pruebas de seguridad se diseñan y realizan para sondear las vulnerabilidades que pueden ocurrir:
  - En el entorno del lado del cliente.
  - En las comunicaciones de red que ocurren cuando se envían los datos entre cliente y servidor.
  - En el entorno del lado del servidor.
- En el lado del cliente, las vulnerabilidades pueden encontrarse en:
  - Errores preexistentes en el navegador
  - Software cliente de correo electrónico
  - Acceso no autorizado a las cookies que se encuentran en el cliente.

# Prueba de Seguridad

- En las comunicaciones de red se pueden encontrar vulnerabilidades como:
  - Los datos pueden ser atacados mediante spoofing, o también con ataques a la integridad.
- En el lado del servidor, las vulnerabilidades incluyen:
  - Ataques de negación de servicio.
  - Incrustación de scripts maliciosos que pueden pasarse entre cliente y servidor.
  - Acceso no autorizado a las bases de datos.

# Prueba de Seguridad

- Para proteger al servidor de éstas vulnerabilidades, se implementan elementos de seguridad:
  - Firewall: bloquea accesos no autorizados.
  - Servicio de autenticación: verificación de la identidad.
  - Encriptación: mecanismo de codificación de datos.
  - Autorización: permite el acceso mediante códigos de autorización apropiados.
- Las pruebas de seguridad deben diseñarse para sondear todas las tecnologías de seguridad con el fin de descubrir huecos en la seguridad.
- Diseñar las pruebas de seguridad requiere amplio conocimiento de los mecanismos y tecnologías de seguridad.

# Prueba de Rendimiento

- Estas pruebas se diseñan para descubrir problemas de rendimiento que pueden resultar por:
  - Falta de recursos en el lado del servidor
  - Ancho de banda inadecuado en la red
  - Capacidades inadecuadas de bases de datos
  - Capacidades deficientes del sistema operativo
  - Funcionalidades de la aplicación web pobremente diseñadas
- Así, éstas pruebas tienen una doble intención:
  1. Evaluar como responde el sistema cuando aumenta la carga de trabajo (número de usuarios, número de transacciones, etc).
  2. Recopilar mediciones que conducirán a modificaciones de diseño para mejorar el rendimiento.

# Prueba de Rendimiento

*Objetivos de la prueba de rendimiento*

- Las pruebas de rendimiento deben simular situaciones de carga del mundo real.
- En la medida que la carga de trabajo crece, la prueba ayudará a responder las siguientes preguntas:
  - ¿El tiempo de respuesta del servidor se degrada a un punto donde es apreciable e inaceptable?
  - ¿En que punto el rendimiento se vuelve inaceptable?
  - ¿Qué componentes del sistema son responsables de la degradación del rendimiento?
  - ¿Cuál es el tiempo promedio de respuesta para los usuarios bajo diferentes condiciones de carga?

# Prueba de Rendimiento

- ¿La degradación del rendimiento afecta a la seguridad?
- ¿Qué sucede cuando se aplican cargas que son mayores que la capacidad máxima del servidor?

- Para esto, se realizan 2 pruebas:

*Prueba de carga*

- Esta prueba examina la carga del mundo real en varios niveles de carga y en varias combinaciones.
- Así, la intención es determinar como responderá la aplicación web y su entorno a varias condiciones de carga, considerando el número de usuarios conectados, transacciones y la carga de datos procesados por transacción

# Prueba de Rendimiento

*Prueba de esfuerzo*

- Esta prueba fuerza aumentar la carga hasta el punto de rompimiento para determinar cuanta capacidad puede manejar el entorno de la aplicación web.
- Se ejercita considerando las mismas variables de la prueba de carga, pero ahora considerando valores límite.

# Fin del curso 2025

**Muchas gracias!!**